

ADP 시계열 분석 학습 대화 전체 기록

사용자

시계열 분석에 대해서 자세히 공부해 보려고 해. ADP 시험을 기준으로 시계열 분석에 대해서 필기, 실기 시험준비를 위해서 알아야 할 사항들에 대해서 정리해 줘

Claude

시계열 분석은 ADP 필기(이론 이해)와 실기(파이썬 구현) 양쪽에서 자주 출제되는 핵심 영역입니다. 학습하시기 편하도록 필기용 이론 정리와 실기용 파이썬 코드 예시를 하나의 문서로 정리해 드리겠습니다.

1. 필기 시험 대비 — 핵심 이론

1-1. 시계열 데이터의 기본 개념

- **시계열(Time Series):** 시간 순서에 따라 관측된 데이터
- **시계열 구성 요소**
 - 추세(Trend): 장기적으로 증가/감소하는 경향
 - 계절성(Seasonality): 일정한 주기(일/주/월/분기/연)로 반복되는 패턴
 - 순환(Cycle): 계절성보다 긴, 불규칙한 주기의 변동 (경기순환 등)
 - 불규칙 요소(Irregular/Noise): 위 요소로 설명되지 않는 무작위 잔차
- **분해 모델**
 - 가법모형(Additive): $Y = T + S + C + I$ — 계절 변동 폭이 일정할 때
 - 승법모형(Multiplicative): $Y = T \times S \times C \times I$ — 계절 변동 폭이 추세에 비례해 커질 때

- **STL 분해(Seasonal-Trend decomposition using Loess):**

국소회귀(Loess) 기반으로 추세·계절성·잔차를 유연하게 분리, 계절 성분의 시간에 따른 변화 허용

1-2. 정상성(Stationarity)

- **정의:** 평균, 분산, 자기공분산이 시간에 따라 변하지 않는 성질(약정상성 기준)

- **정상성이 중요한 이유:** ARMA 계열 모형은 정상 시계열을 가정하므로, 비정상 시계열은 차분·변환을 통해 정상화한 뒤 모델링

- **정상성 검정 방법**

- **ADF 검정(Augmented Dickey-Fuller):** 귀무가설 = "단위근이 존재한다(비정상)". $p\text{-value} < 0.05$ 면 정상성 채택

- **KPSS 검정:** 귀무가설 = "정상성이다" — ADF 와 귀무가설이 반대이므로 함께 사용하면 교차 검증 가능

- **PP 검정(Phillips-Perron):** ADF 의 대안, 이분산·자기상관에 더 강건

- **비정상 → 정상화 방법**

- 차분(Differencing): 1 차 차분 $Y_t - Y_{t-1}$, 계절차분 $Y_t - Y_{t-s}$

- 로그 변환, Box-Cox 변환: 분산 안정화

- 추세 제거(Detrending): 회귀 등으로 추세 성분 제거

1-3. 자기상관 분석

- **ACF(자기상관함수, Autocorrelation Function):** 시차(lag)에 따른 자기 자신과의 상관관계 — 직접 효과 + 간접 효과 모두 포함

- **PACF(편자기상관함수, Partial ACF):** 중간 시차의 영향을 통제한 순수한 시차 효과

- **모형 식별에 활용**

- AR(p): PACF 가 p 시차 이후 절단(cut-off), ACF 는 지수적으로 감소

- MA(q): ACF 가 q 시차 이후 절단, PACF 는 지수적으로 감소

- ARMA(p,q): 둘 다 점진적으로 감소(tail-off)

1-4. 주요 모형

모형	설명
AR(p)	자기회귀 — 과거 p개 관측값의 선형결합으로 현재값 설명
MA(q)	이동평균 — 과거 q개 오차항의 선형결합
ARMA(p,q)	AR + MA 결합, 정상 시계열에 적용
ARIMA(p,d,q)	ARMA에 d차 차분 추가, 비정상 시계열에 적용
SARIMA(p,d,q)(P,D,Q,s)	ARIMA + 계절성분(주기 s)
SARIMAX	SARIMA + 외생변수(exogenous variable)
VAR	다변량 시계열, 여러 변수가 서로 영향을 주고받을 때
지수평활(Exponential Smoothing)	단순지수평활(수준만) → Holt(추세 추가) → Holt-Winters(계절성까지 추가)

● Box-Jenkins 방법론 (ARIMA 모형화 절차)

1. 식별(Identification): 정상성 확인 → 차분 → ACF/PACF로 p, q 후보 결정
2. 추정(Estimation): MLE 등으로 파라미터 추정
3. 진단(Diagnostic Checking): 잔차가 백색잡음인지 검정
4. 예측(Forecasting)

1-5. 모형 진단 및 평가

● 잔차 진단

- 잔차가 백색잡음(White Noise)인지 확인 — 평균 0, 등분산, 자기상관 없음
- **Ljung-Box 검정**: 귀무가설 = "잔차에 자기상관이 없다". p-value > 0.05 면 모델 적합 양호
- Q-Q plot, 잔차 히스토그램으로 정규성 시각 확인

- **모형 선택 기준**

- **AIC(Akaike Information Criterion), BIC(Bayesian Information Criterion)**: 낮을수록 좋음, BIC 가 더 강한 복잡도 페널티
- 여러 (p,d,q) 조합 중 AIC/BIC 최소인 모형 선택 (Grid search)

- **예측 정확도 평가지표**

- MAE(평균절대오차), MSE/RMSE(평균제곱(근)오차), MAPE(평균절대백분율오차)
- MAPE 는 스케일에 무관해 비교에 유리하나, 실제값이 0 에 가까우면 왜곡됨

1-6. 심화 개념 (필기 고득점 대비)

- **VAR(벡터자기회귀)**: 다변량 시계열 간 상호 시차 영향 모델링, 각 변수를 다른 변수들의 과거값으로 회귀
- **그랜저 인과관계(Granger Causality)**: 한 변수의 과거값이 다른 변수의 예측에 통계적으로 유의한 정보를 주는지 검정 (엄밀한 인과관계 아님에 유의)
- **공적분(Cointegration)**: 개별적으로는 비정상이지만 선형결합이 정상인 되는 관계 — Engle-Granger, Johansen 검정
- **상태공간모형(State Space Model)과 칼만필터**: 관측되지 않는 상태변수를 추정, 시변 파라미터 모델링에 활용
- **딥러닝 기반 시계열**: RNN/LSTM/GRU, Prophet(Facebook, 추세+계절성+휴일효과 분해 기반) — 최근 ADP 실기에서 개념 언급 가능성 있음

2. 실기 시험 대비 — 파이썬 구현

2-1. 데이터 준비 및 시각화

```
import pandas as pd
import numpy as np
```

```

import matplotlib.pyplot as plt

# 날짜를 인덱스로 설정
df = pd.read_csv('data.csv', parse_dates=['date'])
df = df.set_index('date')
df = df.asfreq('D') # 빈도 명시 (D: 일, M: 월, Q: 분기 등)

# 시각화
df['value'].plot(figsize=(12, 4), title='Time Series Plot')
plt.show()

# 리샘플링 (예: 일별 -> 월별 평균)
monthly = df['value'].resample('M').mean()

```

2-2. 시계열 분해

```

from statsmodels.tsa.seasonal import seasonal_decompose, STL

# 고전적 분해 (가법/승법)
result = seasonal_decompose(df['value'], model='additive', period=12)
result.plot()
plt.show()

trend = result.trend
seasonal = result.seasonal
resid = result.resid

# STL 분해 (더 유연함)
stl = STL(df['value'], period=12, robust=True)

```

```
res = stl.fit()
res.plot()
```

2-3. 정상성 검정

```
from statsmodels.tsa.stattools import adfuller, kpss

# ADF 검정: 귀무가설 = 비정상(단위근 존재)
adf_result = adfuller(df['value'].dropna())
print(f'ADF Statistic: {adf_result[0]:.4f}')
print(f'p-value: {adf_result[1]:.4f}')
# p-value < 0.05 -> 정상성 (귀무가설 기각)

# KPSS 검정: 귀무가설 = 정상성
kpss_result = kpss(df['value'].dropna(), regression='c')
print(f'KPSS Statistic: {kpss_result[0]:.4f}, p-value: {kpss_result[1]:.4f}')
# p-value < 0.05 -> 비정상 (귀무가설 기각)
```

2-4. 차분 및 ACF/PACF 확인

```
from statsmodels.graphics.tsaplots import plot_acf, plot_pacf

# 1차 차분
df['diff1'] = df['value'].diff().dropna()

# 계절 차분 (주기 12)
df['seasonal_diff'] = df['value'].diff(12).dropna()

# ACF/PACF plot으로 p, q 결정
```

```
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
plot_acf(df['diff1'].dropna(), lags=20, ax=axes[0])
plot_pacf(df['diff1'].dropna(), lags=20, ax=axes[1])
plt.show()
```

2-5. ARIMA / SARIMA 모델링

```
from statsmodels.tsa.arima.model import ARIMA
from statsmodels.tsa.statespace.sarimax import SARIMAX

# ARIMA(p, d, q)
model = ARIMA(df['value'], order=(1, 1, 1))
fitted = model.fit()
print(fitted.summary())

# SARIMAX(p,d,q)(P,D,Q,s) - 계절성 + 외생변수 옵션
sarima_model = SARIMAX(
    df['value'],
    order=(1, 1, 1),
    seasonal_order=(1, 1, 1, 12),
    exog=None # 외생변수 있으면 여기에 지정
)
sarima_fitted = sarima_model.fit(dispatch=False)
print(sarima_fitted.summary())

# 예측
forecast = sarima_fitted.get_forecast(steps=12)
pred_mean = forecast.predicted_mean
conf_int = forecast.conf_int()
```

2-6. 최적 모형 자동 탐색 (auto_arima)

```
# pip install pmdarima
from pmdarima import auto_arima

auto_model = auto_arima(
    df['value'],
    seasonal=True, m=12,
    stepwise=True,
    trace=True,          # 탐색 과정 출력
    suppress_warnings=True,
    information_criterion='aic'
)
print(auto_model.summary())
forecast_auto = auto_model.predict(n_periods=12)
```

2-7. 잔차 진단

```
from statsmodels.stats.diagnostic import acorr_ljungbox

residuals = sarima_fitted.resid

# Ljung-Box 검정: 귀무가설 = 잔차에 자기상관 없음
lb_test = acorr_ljungbox(residuals, lags=[10], return_df=True)
print(lb_test)

# p-value > 0.05 이면 잔차가 백색잡음 -> 모델 적합 양호

# 잔차 시각 진단
```

```
sarima_fitted.plot_diagnostics(figsize=(10, 8))  
plt.show()
```

2-8. 지수평활법 (Holt-Winters)

```
from statsmodels.tsa.holtwinters import ExponentialSmoothing  
  
hw_model = ExponentialSmoothing(  
    df['value'],  
    trend='add',      # 'add' / 'mul' / None  
    seasonal='add',   # 'add' / 'mul' / None  
    seasonal_periods=12  
)  
hw_fitted = hw_model.fit()  
hw_forecast = hw_fitted.forecast(12)
```

2-9. 모델 평가

```
from sklearn.metrics import mean_absolute_error, mean_squared_error  
  
y_true = df['value'][-12:]  
y_pred = forecast_auto[:12]  
  
mae = mean_absolute_error(y_true, y_pred)  
rmse = np.sqrt(mean_squared_error(y_true, y_pred))  
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100  
  
print(f'MAE: {mae:.3f}, RMSE: {rmse:.3f}, MAPE: {mape:.2f}%')
```

```
# AIC/BIC는 모델 fit 결과에서 바로 확인
print(f'AIC: {sarima_fitted.aic:.2f}, BIC: {sarima_fitted.bic:.2f}')
```

2-10. VAR (다변량 시계열)

```
from statsmodels.tsa.api import VAR
from statsmodels.tsa.stattools import grangercausalitytests

# VAR 모델 (여러 열을 가진 DataFrame 필요)
var_model = VAR(df[['var1', 'var2']])
var_fitted = var_model.fit(maxlags=5, ic='aic')
print(var_fitted.summary())

var_forecast = var_fitted.forecast(df[['var1', 'var2']].values[-var_fitted.k_ar:], steps=10)

# 그랜저 인과관계 검정
grangercausalitytests(df[['var1', 'var2']], maxlag=5)
```

3. 실기 시험 실전 팁

1. **문제 유형:** "주어진 데이터로 시계열 예측 모델을 만들고 정확도를 평가하라" 형태가 자주 출제됨

2. 작업 순서 체크리스트

- 데이터 로드 → 결측치/이상치 확인 → 시각화(추세/계절성 파악)
- 정상성 검정(ADF/KPSS) → 필요시 차분
- ACF/PACF로 p,q 후보 확인 (또는 auto_arima로 자동 탐색)
- 모델 적합 → 잔차 진단(Ljung-Box) → 필요시 재조정
- 예측 및 평가지표 산출(RMSE, MAPE 등) → 결과 해석 서술

3. 자주 실수하는 부분

- seasonal_decompose 는 주기(period)를 반드시 명시해야 함
- 차분 후 첫 값은 NaN 이 생기므로 .dropna() 필수
- SARIMAX 의 seasonal_order 의 s(주기)를 데이터 특성에 맞게 설정(월별=12, 분기=4, 주간=52 등)
- train/test 분할 시 시계열 특성상 랜덤 셔플 금지 — 시간 순서 유지한 채로 분할

4. 서술형 답안 작성 시: 단순 수치 나열보다 "ADF 검정 결과 p-value가 X이므로 정상성을 만족(또는 불만족)하여 Y차 차분을 적용함" 처럼 **검정 결과 → 판단 → 조치**의 논리 흐름으로 서술하면 채점에 유리함

4. 추가 학습 자료 방향

- 통계청/한국데이터산업진흥원 ADP 기출문제 및 가이드북에서 시계열 파트 기출 유형 확인
- statsmodels 공식 문서의 Time Series Analysis 섹션
- Box, Jenkins, Reinsel 의 *Time Series Analysis: Forecasting and Control* (고전 이론서, 필기 심화용)

필기 이론과 실기 파이썬 코드를 함께 정리해 드렸습니다. 구성은 다음과 같습니다.

필기 파트: 시계열 구성요소, 정상성과 검정법(ADF/KPSS), ACF/PACF, ARIMA 계열 모형, Box-Jenkins 방법론, 모형 평가지표, VAR·그랜저 인과관계 같은 심화 개념까지 포함했습니다.

실기 파트: statsmodels, pmdarima 등을 활용한 정상성 검정, 시계열 분해, ARIMA/SARIMAX 적합, 잔차 진단, 예측 평가까지 바로 실행 가능한 코드 예시로 정리했습니다.

실기 시험은 특히 "정상성 검정 → 판단 → 차분 → 모델링 → 잔차 진단"의 흐름을 코드와 함께 논리적으로 서술하는 연습이 중요하니, 문서 마지막의 실전 팁 부분을 참고하시면 좋겠습니다.

사용자

가법모형과 승법모형에서 각 항목을 더하거나 곱하는 것의 의미가 뭐야? 더 하거나 곱해서 뭘 어떻게 하겠다는거야?

Claude

좋은 질문입니다. 이 부분은 개념적으로 헷갈리기 쉬운데, "실제 관측값을 어떻게 분해해서 재구성하느냐"의 문제로 이해하시면 쉽습니다.

핵심 아이디어

시계열의 실제 관측값 Y 를 추세(T), 계절성(S), 불규칙 요소(I)로 **분해**했다가, 그 성분들을 다시 **결합**하면 원래 값 Y 가 나와야 합니다. 이때 "어떻게 결합해야 원래 값이 복원되는가"가 가법이나 승법을 결정합니다.

가법모형: $Y = T + S + I$

의미: 계절적으로 오르내리는 폭이 추세 수준과 무관하게 **일정한 절대량**이라고 가정
예를 들어 어떤 상품의 월 매출이:

- 1 월(비수기): 100 만원
- 7 월(성수기): 130 만원 → 계절 효과가 **+30 만원**

이 회사가 성장해서 몇 년 후 추세가 2배(200만원 수준)가 되었을 때도:

- 1 월: 200 만원
- 7 월: 230 만원 → 여전히 계절 효과가 **+30 만원**

→ 이렇게 추세 크기와 상관없이 계절 변동폭이 **더해지는 절대값(고정폭)**으로 일정하게 유지될 때 가법모형이 적합합니다.

승법모형: $Y = T \times S \times I$

의미: 계절적으로 오르내리는 폭이 추세 수준에 **비례해서 커지거나 작아진다고** 가정
같은 예시를 승법으로 보면:

- 1 월: 100 만원, 7 월: 130 만원 → 계절 효과가 **평소의 1.3 배**(30% 증가)
회사가 성장해서 추세가 2배(200만원)가 되면:
- 1 월: 200 만원, 7 월: **260 만원** (여전히 1.3 배, 절대량은 +60 만원으로 커짐)
→ 매출 규모가 커질수록 계절적 변동폭(절대량)도 함께 커지는 경우, 즉 성분들이 **비율(배수)로 곱해지는 관계**일 때 승법모형이 적합합니다.

실무에서 구분하는 방법

시계열 그래프를 그려봤을 때

- 계절 변동의 진폭(파도의 높이)이 **시간이 지나도 일정** → 가법모형
 - 계절 변동의 진폭이 **추세를 따라 점점 커지거나 작아짐** (나팔 모양) → 승법모형
- 또한 승법모형은 로그를 취하면 가법모형으로 바꿀 수 있습니다.

$$\log(Y) = \log(T \times S \times I) = \log(T) + \log(S) + \log(I)$$

이 성질 때문에 실무에서는 "승법 관계가 의심되면 로그 변환 후 가법모형으로 분석"하는 방식도 많이 사용합니다. statsmodels의 `seasonal_decompose(model='multiplicative')`도 내부적으로 이와 유사한 방식으로 처리합니다.

사용자

승법 관계가 의심되면 로그 변환 후 가법모형으로 분석한다는게 무슨 뜻이야? 여기서 의심된다는 것의 의미는 뭐고 그래서 변환해서 분석해서 어떻게 한다는 거야?

Claude

좋은 질문입니다. 하나씩 풀어서 설명해 드리겠습니다.

"승법 관계가 의심된다"는 것의 의미

여기서 "의심된다"는 건 데이터를 보고 "이 시계열은 아무래도 승법모형($Y = T \times S \times I$) 구조인 것 같다"는 **판단의 근거가 있다**는 뜻입니다. 구체적으로는:

1. **그래프를 그려봤을 때**: 시간이 지날수록(추세가 커질수록) 계절 변동의 진폭(파도의 높이)이 함께 커지는, **나팔 모양(퍼지는 모양)**의 패턴이 보임
 2. **통계적으로**: 시계열의 **분산이 수준(level)에 비례해서 커지는 이분산성(heteroscedasticity)**이 관찰됨 — 값이 클 때는 변동폭도 크고, 값이 작을 때는 변동폭도 작음
- 즉 "의심된다"는 건 육안 확인이든 통계적 진단이든, "이 데이터는 성분들이 더해지는 게 아니라 곱해지는 관계로 보인다"는 근거가 있다는 뜻입니다.

로그 변환 후 가법모형으로 분석한다는 것

목적: ARIMA, 회귀분석, 지수평활 등 대부분의 시계열 기법은 가법적 구조(더해지는 관계)를 가정하고 만들어져 있습니다. 승법 구조인 데이터를 억지로 가법모형에 넣으면 잔차가 이분산성을 가져서 검정이나 신뢰구간이 왜곡됩니다. 그래서 **승법 구조를 가법 구조로 바꿔주는 작업**이 필요한데, 그 방법이 로그 변환입니다.

원리: 로그의 성질상 곱셈이 덧셈으로 바뀝니다.

$$Y = T \times S \times I$$

$$\log(Y) = \log(T) + \log(S) + \log(I)$$

즉 원래 데이터에 로그를 씌우기만 하면, 곱셈 관계였던 성분들이 덧셈 관계로 바뀝니다. 이러면 "로그를 씌운 데이터"에 대해서는 평범한 가법모형(ARIMA 등)을 그대로 적용할 수 있게 됩니다.

실제 분석 절차

1. 원본 데이터 Y에 로그를 취해 $\log(Y)$ 를 만듦
2. $\log(Y)$ 에 대해 정상성 검정(ADF), 차분, ARIMA/SARIMA 적합 등 **일반적인 가법 기반 절차**를 그대로 진행
3. 예측값도 로그 스케일로 나옴 (예: $\log(\hat{Y})$)
4. **마지막에 지수변환(exp)을 취해 원래 스케일로 복원**: $\hat{Y} = \exp(\log(\hat{Y}))$

```
import numpy as np
```

```

from statsmodels.tsa.arima.model import ARIMA

# 1. 로그 변환
log_y = np.log(df['value'])

# 2. 로그 스케일에서 가법모형(ARIMA)으로 분석
model = ARIMA(log_y, order=(1,1,1))
fitted = model.fit()

# 3. 예측 (로그 스케일)
log_forecast = fitted.forecast(steps=12)

# 4. 원래 스케일로 복원
forecast = np.exp(log_forecast)

```

정리하면

단계	하는 일	이유
진단	그래프/분산 확인	승법 구조인지 판단 (=의심)
변환	$\log(Y)$ 계산	곱셈 관계를 덧셈 관계로 변환
분석	로그 데이터에 가법모형 적용	표준 도구(ARIMA 등)를 그대로 쓸 수 있게 함
복원	$\exp()$ 로 역변환	사람이 이해할 수 있는 원래 단위로 되돌림

한 가지 참고하실 점은, 로그 변환된 값으로 차분을 하면 그 차분값이 근사적으로 **"변화율(%)"**을 의미하게 된다는 것입니다($\log(Y_t) - \log(Y_{t-1}) \approx \text{증가율}$). 그래서 로그차분 시계열은 경제·금융 데이터의 수익률 분석에서도 자주 사용됩니다.

사용자

STL 분해(Seasonal-Trend decomposition using Loess): 국소회귀(Loess) 기반으로 추세·

계절성·잔차를 유연하게 분리, 계절 성분의 시간에 따른 변화 허용

에서 국소회귀(Loess)가 뭐고, 추세·계절성·잔차를 유연하게 분리, 계절 성분의 시간에 따른 변화 허용 이 말의 의미도 뭔지 모르겠어. 자세히 설명해 줘

 Claude

좋은 질문입니다. 이 부분은 기존 고전적 분해법(seasonal_decompose)과 비교해서 이해하면 훨씬 명확해집니다.

1. 국소회귀(Loess)란?

Loess(Locally Estimated Scatterplot Smoothing)는 데이터 전체에 하나의 수식(직선이나 곡선)을 딱 맞추는 게 아니라, **각 지점 근처의 데이터만 이용해서 그 지점의 값을 부드럽게 추정하는** 방법입니다.

일반 회귀 vs 국소회귀 비교

일반 회귀(예: 선형회귀)

- 전체 데이터에 대해 " $y = ax + b$ " 같은 **하나의 고정된 직선**을 찾음
- 데이터 전체를 관통하는 단일한 규칙

국소회귀(Loess)

- 특정 지점 x_0 근처의 데이터(예: 앞뒤 몇 개 점)만 골라서, 그 근처에서만 성립하는 작은 회귀선을 만들고 x_0 에서의 값을 추정
- 이 과정을 x_0 를 옆으로 옮겨가며 데이터 전체에 대해 반복
- 결과적으로 **구간마다 다른 모양을 반영하는 부드러운 곡선**이 만들어짐

비유: 일반 회귀가 "산맥 전체를 관통하는 하나의 직선 자를 대는 것"이라면, 국소회귀는 "창문을 조금씩 옆으로 옮겨가며 창문 안에 보이는 부분만 보고 그때그때 부드럽게 선을 그리는 것"입니다. 그래서 전체적으로 봤을 때 구불구불한 실제 흐름을 유연하게 따라갈 수 있습니다.

2. "추세·계절성·잔차를 유연하게 분리"의 의미

기존 고전적 분해법(seasonal_decompose)은 추세를 구할 때 **이동평균(moving average)**을 사용합니다. 이동평균은:

- 계산이 단순하지만, 추세가 급격히 꺾이는 구간에서는 둔감하게 반응(뭉개짐)
- 데이터 양 끝(맨 앞/맨 뒤)에서는 값을 계산할 수 없어 결측치가 생김

반면 STL은 추세와 계절 성분 각각을 구할 때 **국소회귀(Loess)**를 반복 적용합니다.

- 추세: 데이터를 구간별로 국소적으로 부드럽게 스무딩 → 급격한 변화도 유연하게 따라감, 끝부분도 추정 가능
- 계절성: 같은 시점(예: 매년 1 월끼리)들을 모아서 그 안에서 국소회귀를 적용해 계절 패턴 추정
- 이 과정을 추세 ↔ 계절성 사이를 오가며 반복(iterate)해서 서로의 추정치를 계속 다듬어감

즉 "유연하다"는 건 정해진 딱딱한 공식(고정폭 이동평균) 대신, **데이터 각 구간의 실제 모양에 맞춰 곡선을 국소적으로 다시 그려가며 성분을 뽑아낸다는** 뜻입니다.

3. "계절 성분의 시간에 따른 변화 허용"의 의미

이게 STL의 가장 큰 차별점입니다.

고전적 분해법의 가정: 계절 패턴이 **매년 완전히 똑같다**고 가정

- 예: "7 월은 항상 1 월보다 정확히 +30 만원 많다"는 계절 효과가 10 년 내내 고정
- 실제로는 시간이 지나며 계절 패턴 자체가 서서히 바뀌는 경우가 많은데(예: 에어컨 보급 확대로 여름 전력 수요의 계절성 자체가 커짐), 이런 변화를 반영 못 함

STL의 가정: 계절 패턴이 **서서히 변할 수 있다**고 허용

- "매년 1 월"들끼리 모아서 국소회귀를 적용할 때, 옛날 1 월과 최근 1 월에 서로 다른 가중치를 주면서 부드럽게 추정
- 그 결과 1 년차의 7 월 계절효과와 10 년차의 7 월 계절효과가 완전히 같지 않고, 조금씩 다르게(트렌드를 따라 변화하며) 추정될 수 있음

정리 비교표

항목	고전적 분해(이동평균)	STL(Loess 기반)
추세 추정 방식	고정폭 이동평균	국소회귀(구간별 유연한 곡선)
계절 패턴	전체 기간 동안 완전히 고정	시간에 따라 서서히 변화 허용
데이터 끝부분	결측치 발생	추정 가능
이상치에 대한 강건성	약함	robust=True 옵션으로 이상치 영향 완화 가능

코드로 보면 이 차이가 명확히 보입니다.

```
from statsmodels.tsa.seasonal import STL

stl = STL(df['value'], period=12, robust=True)
res = stl.fit()

res.seasonal.plot(title='계절 성분') # 이 성분을 연도별로 겹쳐 그려보면
# 완전히 동일하지 않고 조금씩 모양이 달라지는 것을 확인 가능
```

사용자

정상성이 정의: 평균, 분산, 자기공분산이 시간에 따라 변하지 않는 성질라고 했는데 시간에 따라 변하지 않으면 그게 뭘 의미하는거야?

Claude

좋은 질문입니다. "시간에 따라 변하지 않는다"는 게 추상적으로 느껴질 수 있는데, 각 조건이 실제로 어떤 상황을 의미하는지 하나씩 풀어보겠습니다.

핵심 아이디어

정상성이란 시계열을 어느 구간에서 잘라서 봐도 통계적 성질이 비슷비슷하다는 뜻입

니다. 예를 들어 1~12월 구간을 봐도, 50~61개월 구간을 봐도, 그 구간의 "평균적인 수준", "흔들리는 정도", "옆 시점과의 관계"가 서로 크게 다르지 않다는 것입니다.

조건별로 풀어보기

① 평균이 시간에 따라 변하지 않는다

의미: 시계열이 어느 시점을 중심으로 오르내리든, 그 **중심 수준(level)** 자체는 시간이 지나도 같은 자리에 머물러 있어야 함

- **정상성 위반 예:** 회사 매출이 매년 꾸준히 증가하는 추세(trend)를 가진 경우 → 초반 구간의 평균은 낮고, 후반 구간의 평균은 높음 → 평균이 시간에 따라 변함 → **비정상**
- **정상 예:** 어떤 공정에서 매일 측정한 온도가 20 도를 중심으로 위아래로 흔들릴 뿐, 시간이 지나도 계속 20 도 근처에서 맴돌 → 평균 일정 → 이 조건은 만족

② 분산이 시간에 따라 변하지 않는다

의미: 데이터가 평균을 중심으로 **얼마나 크게 흔들리는지(변동 폭)**가 시간이 지나도 일정해야 함

- **정상성 위반 예:** 주가 데이터에서 초반에는 가격이 조금씩만 움직이다가, 후반으로 갈수록 변동성이 커지는 경우(변동폭이 나팔모양으로 퍼짐) → 분산이 시간에 따라 커짐 → **비정상**
- **정상 예:** 매일 측정한 온도의 흔들림 정도가 여름이든 겨울이든 항상 비슷한 폭으로 오르내림 → 분산 일정

③ 자기공분산이 시간에 따라 변하지 않는다 (시차에만 의존)

이게 가장 헷갈리는 부분인데, 풀어보면 이렇습니다.

자기공분산이란 "특정 시점의 값과, 거기서 k만큼 떨어진 시점의 값이 서로 얼마나 비슷하게 움직이는지"를 나타내는 지표입니다.

조건의 의미: 이 관계가 "몇 시점을 기준으로 재는지(t)"가 아니라, "얼마나 떨어져 있는지(시차 k)"에만 의존해야 함

- 예를 들어 1 월과 2 월의 관계(시차=1)가, 50 개월째와 51 개월째의 관계(시차=1)와 **비슷한 강도**여야 함
- 즉 "어디서 재느냐"는 중요하지 않고, "얼마나 떨어져서 재느냐(시차)"만 중요함
- **정상성 위반 예:** 계절성이 있는 데이터에서 계절 패턴 자체가 해마다 점점 강해지는 경우(에어컨 보급 확대로 여름-겨울 온도차 관련 전력수요 격차가 매년 커짐) → 같은 시차(예: 12 개월 전)와의 관계 강도가 초반보다 후반에 더 강해짐 → 비정상
- **정상 예:** 백색잡음(순수한 무작위 노이즈)처럼 어제 값과 오늘 값의 관계가 언제 재도 항상 "관계없음(0)"으로 일정 → 정상

왜 이 세 가지가 다 중요한가

세 조건을 합치면 결국 이런 뜻입니다: **시계열의 어느 한 구간(예: 1~50번째 관측치)의 통계적 특성을 보고 다른 구간(예: 100~150번째)의 특성을 그대로 짐작해도 무방하다.**

이게 왜 중요하냐면, ARIMA 같은 모형은 "과거 패턴이 미래에도 똑같이 반복된다"는 전제로 하나의 수식(파라미터)을 추정합니다. 만약 평균이나 분산, 자기상관 구조가 시간에 따라 계속 바뀐다면, 과거 구간에서 추정한 파라미터가 미래 구간에는 안 맞을 수 있습니다. 그래서 모델링 전에 **차분이나 로그변환 등으로 이 세 가지를 최대한 "시간에 무관하게" 만들어주는 작업(정상화)**을 하는 것입니다.

시각적으로 구분하는 팁

그래프 모양	위반되는 조건
우상향/우하향하는 직선 형태 (추세)	평균
시간이 지날수록 흔들림 폭이 커지는 나팔 모양	분산
일정한 주기로 오르내리는 패턴 (계절성)이 반복	평균 (주기적으로 평균이 오르내림)
특정 구간에서만 상관관계 패턴이 갑자기 달라짐	자기공분산

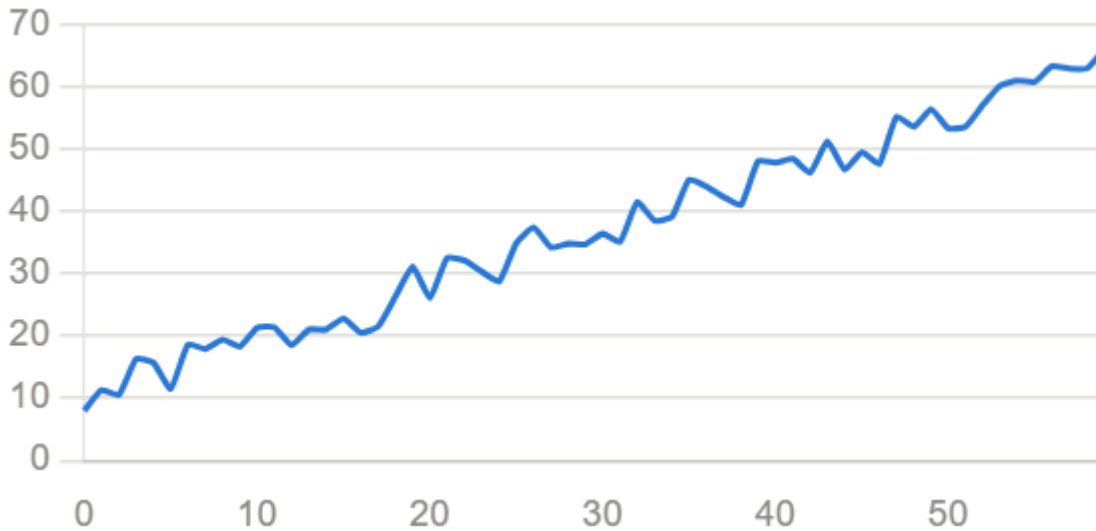
사용자

시각적으로 구분하는 팁과 관련된 실제 이미지를 보여줄 수 있어?

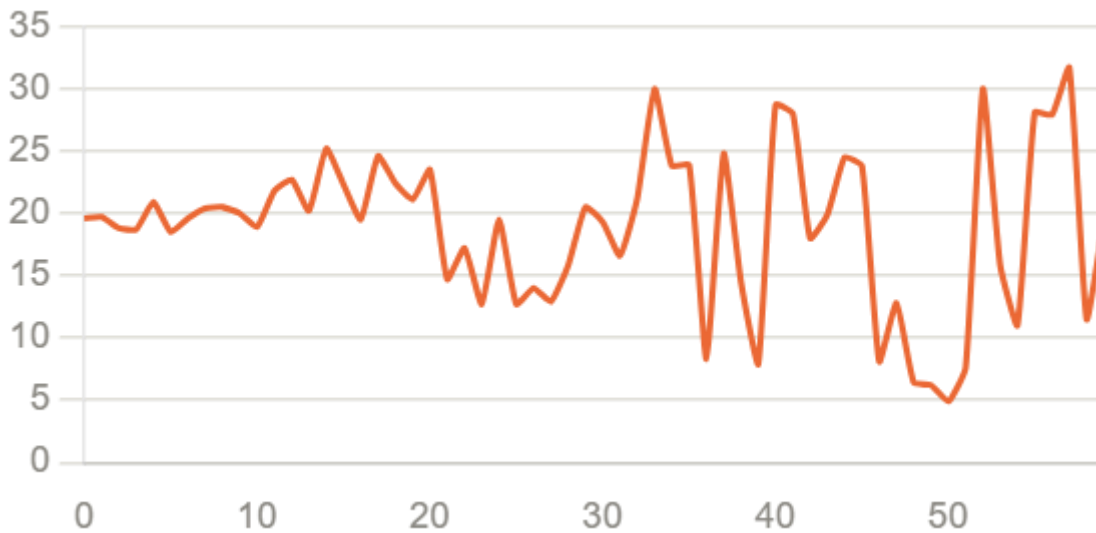


[인터랙티브 시각자료 표시됨: "정상성 위반 사례 4가지" — 4개의 라인 차트를 2x2로 배치]

1. 평균이 변함 (추세): 시간이 지날수록 중심 수준이 계속 상승하는 우상향 그래프



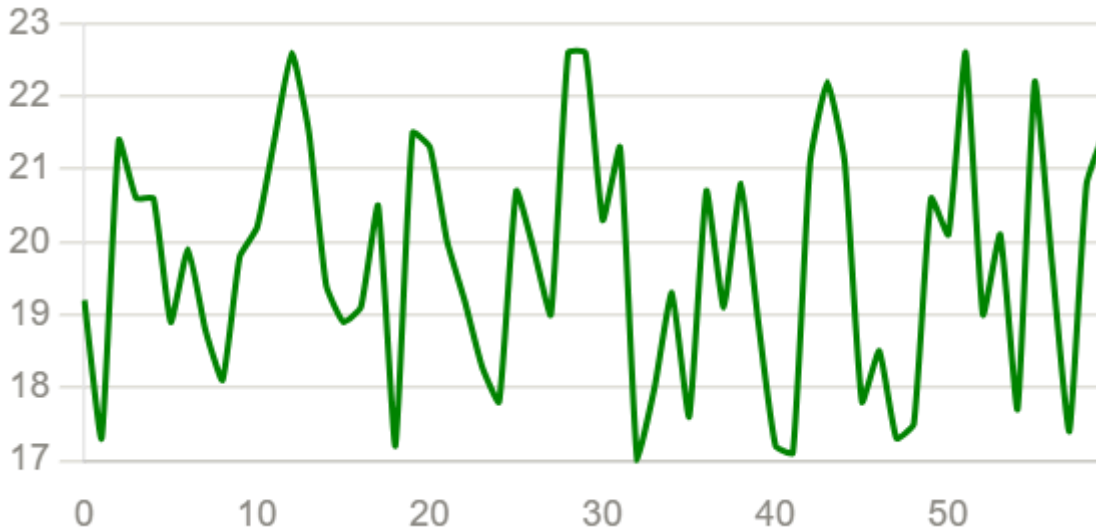
2. 분산이 변함 (이분산성): 시간이 지날수록 흔들림 폭이 나팔모양으로 커지는 그래프



3. 계절성 (주기적 평균 변화): 일정 주기로 평균 수준이 오르내림을 반복하는 파형 그래프



4. 정상 시계열: 평균·분산·자기공분산이 시간에 따라 일정하게 유지되는 그래프



4가지 예시를 비교해서 보시면 각 조건이 위반될 때 그래프가 어떤 모양으로 나타나는지 명확히 구분되실 겁니다.

- **평균 위반(추세):** 선이 전체적으로 우상향 — 중심 수준 자체가 이동
- **분산 위반(이분산성):** 뒤로 갈수록 흔들림 폭이 커지는 나팔 모양
- **계절성:** 일정한 주기로 오르내림이 반복되는 파형
- **정상 시계열:** 일정한 수준을 중심으로 비슷한 폭으로 계속 흔들림

실제 데이터를 다룰 때는 이렇게 눈으로 먼저 패턴을 확인한 뒤, ADF/KPSS 검정으로 수

치적으로 재확인하는 순서로 진행하시면 됩니다.

사용자

비정상 시계열은 차분·변환을 통해 정상화한 뒤 모델링한다고 했는데 차분/변환을 하면 그 데이터가 변하게 되는데 그렇게 변환한 데이터를 모델링하고 나서 나중에 해석할때 그냥 그대로 해석하면 원래 변환전 데이터의 성질이 제대로 설명되는거야?

Claude

좋은 질문입니다. 결론부터 말씀드리면 **"그대로 해석하면 안 되고, 반드시 원래 스케일로 되돌리거나 해석 방식을 바꿔야 한다"**가 맞습니다. 왜 그런지, 그리고 실제로 어떻게 처리하는지 설명해 드리겠습니다.

핵심 문제: 변환된 데이터의 "단위"가 원본과 다르다

차분이나 로그변환을 하면 데이터의 **의미(단위)** 자체가 바뀝니다.

변환	원본의 의미	변환 후 값의 의미
원본 Y	특정 시점의 수준(level) — 예: 매출 100만원	—
1차 차분 $Y_t - Y_{t-1}$	—	전월 대비 변화량 (절대적 증감액) — 예: +5만원
로그차분 $\log(Y_t) - \log(Y_{t-1})$	—	전월 대비 변화율(%) — 예: +5%

즉 원래 데이터는 "얼마인가(수준)"를 나타내지만, 차분된 데이터는 "얼마나 변했는가 (변화량/변화율)"를 나타냅니다. 완전히 다른 질문에 대한 답입니다. 그래서 모델을 차분된 데이터로 학습시켰다면, 그 모델이 예측하는 것도 "다음 시점의 값"이 아니라 "다음 시점까지의 변화량"입니다.

그래서 실제로 어떻게 처리하나

1) 예측(forecast) 목적이라면 → 역변환(inverse transform)해서 원래 스케일로 복원

모델이 뱉어내는 예측값은 차분/변환된 스케일이므로, 사람이 보는 "매출이 얼마일 것이다" 같은 해석을 하려면 **반드시 역연산으로 복원**해야 합니다.

- 로그변환 했다면 → `exp()`로 복원
- 1 차 차분 했다면 → 마지막 관측값에 예측된 변화량을 **누적해서 더해줌**

1차 차분된 데이터로 예측했을 때 원래 스케일 복원

```
last_value = df['value'].iloc[-1]
```

```
diff_forecast = fitted_diff_model.forecast(steps=5) # 차분값 예측
```

```
level_forecast = last_value + diff_forecast.cumsum() # 누적합으로 원래 스케일 복원
```

참고로 statsmodels의 ARIMA(order=(p, d, q))처럼 **d(차분 차수)**를 모델 안에 지정하면, 라이브러리가 내부적으로 차분해서 학습한 뒤 예측 시 **자동으로 역차분(un-differencing)까지 해서** 원래 스케일의 예측값을 돌려줍니다. 그래서 직접 `.diff()`를 해서 별도 모델에 넣지 않는 한, 보통은 이 부분을 신경 쓰지 않아도 됩니다.

2) 계수(coefficient)나 관계 해석이 목적이라면 → 해석 자체를 변환된 단위 기준으로 바꿔서 말함

예측이 아니라 "이 변수가 매출에 미치는 영향" 같은 것을 해석할 때는, 역변환하지 않고 **변환된 변수 자체의 의미로 해석 문장을 바꿔서** 말해야 합니다.

- 원본 스케일 회귀계수: "광고비가 1 만원 늘면 매출이 3 만원 늘어난다"
- 로그변환된 종속변수($\log(Y)$)로 회귀했다면: "광고비가 1 단위 늘면 매출이 **약 3% 증가**한다" (로그모형에서 계수는 대략 %변화로 해석)
- 1 차 차분된 데이터로 회귀했다면: "이 변수는 매출의 **수준이 아니라 매출의 변화(증감폭)**에 영향을 준다"는 식으로 해석 대상 자체가 달라짐

즉 통계 결과지에 나온 숫자를 원본 단위인 것처럼 그대로 읽으면 틀린 해석이 되고, "**이 숫자는 로그 스케일/차분 스케일에서의 의미다**"라는 걸 명시하고 그에 맞게 재해석해야 합니다.

3) 정확도 평가(RMSE, MAPE 등)는 반드시 원래 스케일로 복원한 뒤 계산

만약 차분/로그 스케일에서 그대로 RMSE를 계산하면, 그 오차가 실제 비즈니스 단위

(예: 원화 매출)로 얼마나 틀렸는지 알 수 없습니다. 그래서 실무에서는:

1. 모델은 정상화된(차분/로그) 데이터로 학습
2. 예측값을 **역변환해서 원래 스케일로 복원**
3. 실제 관측값(원래 스케일)과 비교해서 RMSE, MAPE 등을 계산

잘못된 방식: 로그 스케일에서 바로 RMSE 계산 (해석 불가능한 단위)

```
rmse_wrong = np.sqrt(mean_squared_error(log_y_true, log_y_pred))
```

올바른 방식: 원래 스케일로 복원 후 계산

```
y_true_original = np.exp(log_y_true)
```

```
y_pred_original = np.exp(log_y_pred)
```

```
rmse_correct = np.sqrt(mean_squared_error(y_true_original, y_pred_original))
```

정리

목적	처리 방법
미래 값 예측(사람이 볼 결과)	역변환(지수/누적합)해서 원래 스케일로 복원
계수·관계 해석	역변환 없이, 해석 문장 자체를 "변화율/변화량" 기준으로 바꿔서 서술
모델 정확도 평가	반드시 원래 스케일로 복원한 뒤 오차지표 계산
잔차 진단(Ljung-Box 등)	차분/변환된 스케일 그대로 진단 (정상성 확보된 상태에서 봐야 하므로 이 경우는 변환 후 데이터가 맞음)

즉 "정상화를 위한 변환"과 "결과를 사람이 이해하기 위한 해석"은 별개의 단계이고, 후자에서는 항상 "이 숫자가 어떤 스케일/단위에 있는가"를 의식하고 필요하면 되돌려서 봐야 원본 데이터의 성질이 제대로 설명됩니다.

사용자

단위근이 존재한다(비정상) 라고 했는데 단위근이 뭐고 이게 존재하면 왜 비정상(정상성이 아닌거지?)인지 알려줘



단위근(Unit Root) 개념은 AR(1) 모형이라는 가장 단순한 모형으로 살펴보면 명확하게 이해되실 겁니다.

AR(1) 모형으로 보는 단위근

가장 단순한 자기회귀 모형을 생각해 보겠습니다.

$$Y_t = \phi \cdot Y_{t-1} + \varepsilon_t$$

여기서 ϕ (파이)는 "어제 값이 오늘 값에 얼마나 영향을 주는가"를 나타내는 계수이고, ε_t 는 무작위 충격(백색잡음)입니다.

이 식을 "특성방정식(characteristic equation)"이라는 형태로 바꿔서 쓰면:

$$(1 - \phi z) = 0 \rightarrow z = 1/\phi$$

이 방정식의 **근(root)**이 **$z = 1$** 인 경우를 "단위근을 가진다"고 부릅니다. 즉 $1/\phi = 1$ 이 되려면 **$\phi = 1$** 이어야 합니다.

ϕ 값에 따라 무슨 일이 일어나는가

$\phi < 1$ (정상 시계열, 단위근 없음)

$Y_t = 0.7 \cdot Y_{t-1} + \varepsilon_t$ 라고 하면, 오늘 발생한 충격 ε_t 의 영향이 다음 시점엔 0.7배, 그 다음엔 0.49배... 로 **점점 줄어들며 사라집니다**. 즉 어떤 충격이 와도 시간이 지나면 원래의 평균 수준으로 **되돌아가는 힘(평균회귀, mean reversion)**이 있습니다.

$\phi = 1$ (단위근 존재, 비정상 $\rightarrow$ 랜덤워크)

$Y_t = 1 \cdot Y_{t-1} + \varepsilon_t$ 가 되는데, 이게 바로 **랜덤워크(random walk)**입니다. 오늘 받은 충격 ε_t 는 줄어들지 않고 **다음 시점, 그 다음 시점... 영원히 그대로 남아서 누적됩니다**.

$$Y_t = Y_0 + \varepsilon_1 + \varepsilon_2 + \varepsilon_3 + \dots + \varepsilon_t \quad (\text{모든 과거 충격이 소멸 없이 누적됨})$$

왜 이게 "비정상"을 의미하는가

이 누적 구조 때문에 정상성의 조건들이 깨집니다.

분산이 시간에 따라 계속 커짐: 위 식에서 $\text{Var}(Y_t) = t \cdot \sigma^2$ 가 됩니다(각 충격의 분산이 σ^2 이고, t 개가 누적되므로). 즉 **시간이 지날수록 분산이 무한정 커짐** → 정상성의 "분산이 일정해야 한다"는 조건 위반

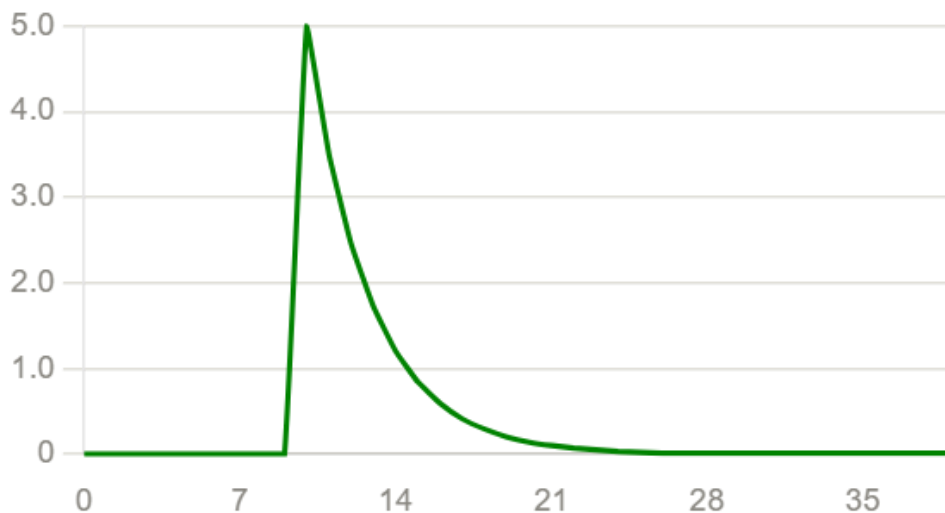
평균회귀가 없음 → 충격이 영구적: $\phi < 1$ 일 때는 어떤 충격이 와도 결국 원래 수준으로 돌아오지만, $\phi = 1$ 이면 충격이 온 지점에서 **영구적으로 눌러앉아** 버립니다. 그래서 시계열이 특정 수준 근처에 머물지 않고 **어디로든 정처 없이 떠돌아다니는(wander)** 모습을 보입니다 — 이게 바로 "평균이 시간에 따라 변한다"는 조건 위반과 직결됩니다.

시각적으로 비교하면

같은 크기의 충격 하나를 10번째 시점에 줬을 때, $\phi = 0.7$ 인 정상 시계열은 그 충격이 서서히 사라지지만, $\phi = 1$ 인 단위근 시계열은 그 충격이 사라지지 않고 그 이후 수준 자체를 영구히 밀어 올립니다. 그래프로 직접 비교해서 보여드리겠습니다.

[인터랙티브 시각자료 표시됨: "10번째 시점에 동일한 충격을 준 뒤 반응 비교" — 2개의 라인 차트]

1. $\phi = 0.7$ (정상 시계열): 충격이 서서히 사라지고 원래 수준(0)으로 회귀하는 그래프



2. $\phi = 1$ (단위근, 랜덤워크): 충격이 사라지지 않고 영구적으로 새로운 수준으로 올라선 채 유지되는 그래프



두 그래프 모두 10번째 시점에서 똑같은 크기의 충격을 한 번 줬는데, 이후 반응이 완전히 다릅니다. $\phi=0.7$ 은 충격이 지수적으로 사그라들며 원래 수준(0)으로 돌아오지만, $\phi=1$ 은 그 충격 크기만큼 새로운 수준으로 영구히 올라선 채 유지됩니다.

ADF 검정이 하는 일

ADF(Augmented Dickey-Fuller) 검정은 정확히 " **ϕ 가 1인가, 1보다 작은가**"를 통계적으로 판정하는 검정입니다.

- 귀무가설(H_0): $\phi = 1$ (단위근 존재 → 비정상)
- 대립가설(H_1): $\phi < 1$ (단위근 없음 → 정상)

p-value가 작으면(보통 0.05 미만) 귀무가설을 기각 → "단위근이 없다, 즉 정상성을 만족한다"고 판단하는 것입니다.

실무적 대응: 차분을 하면 왜 해결되는가

단위근이 있는 시계열($Y_t = Y_{t-1} + \epsilon_t$)을 1차 차분하면:

$$Y_t - Y_{t-1} = \epsilon_t$$

우변이 그냥 백색잡음 ϵ_t 만 남습니다. 백색잡음은 평균 0, 분산 일정, 자기상관 없음 —

정의상 정상 시계열입니다. 이게 바로 "단위근이 있는 비정상 시계열은 차분을 하면 정상화된다는 원리의 수학적 근거입니다. ARIMA(p, **d**, q)에서 **d**가 바로 "몇 번 차분해야 단위근이 사라지는가"를 나타내는 값입니다.

사용자

비정상 → 정상화 방법

- 차분(Differencing): 1 차 차분 $Y_t - Y_{t-1}$, 계절차분 $Y_t - Y_{t-s}$
- 로그 변환, Box-Cox 변환: 분산 안정화
- 추세 제거(Detrending): 회귀 등으로 추세 성분 제거

라고 했는데 이것들이 왜 정상화를 시키는거야?

Claude

좋은 질문입니다. 세 가지 방법이 각각 "정상성의 어떤 조건을 어떻게 고쳐주는지" 정확히 대응시켜서 보면 이해가 쉽습니다. 이전에 설명드린 단위근 개념과도 바로 연결됩니다.

1. 차분(Differencing) — 확률적 추세(단위근)를 제거

이건 방금 전에 설명드린 것과 정확히 같은 원리입니다.

$Y_t = Y_{t-1} + \varepsilon_t$ (단위근 존재, 랜덤워크)와 같은 구조를 가진 시계열은, 매 시점마다 과거 충격들이 **소멸하지 않고 누적**되기 때문에 분산이 계속 커지고, 평균도 정처 없이 떠돕니다.

여기에 1차 차분을 적용하면:

$$Y_t - Y_{t-1} = \varepsilon_t$$

좌변이 곧 우변의 백색잡음 ε_t 와 같아집니다. 백색잡음은 정의 자체가 "평균 0, 분산 일정, 자기상관 없음"이므로 **자동으로 정상성의 세 조건을 모두 만족**합니다. 즉 차분은 "누적되고 있던 충격의 총합"에서 "새로 추가된 한 시점의 충격"만 남기는 연산이기 때

문예, 시간에 따라 계속 불어나던 성질(비정상성의 원인)을 제거해줍니다.

계절차분($Y_t - Y_{t-s}$)도 동일한 논리인데, 다만 "1시점 전"이 아니라 "같은 계절의 s시점 전"과 비교합니다. 계절성 때문에 매년 같은 달끼리 값이 비슷하게 오르내리는 구조(계절 단위근이라고도 함)를 제거해서, 계절 패턴이 반복되며 만들던 주기적인 평균 변화를 없애줍니다.

2. 로그/Box-Cox 변환 — 분산이 수준에 비례해서 커지는 문제 해결

이건 차분과는 성격이 다른 문제를 해결합니다. 차분이 "평균이 떠도는 문제"를 해결한다면, 로그 변환은 "**분산이 수준(level)에 비례해서 커지는 문제(이분산성)**"를 해결합니다.

승법 구조($Y = T \times S \times I$)를 가진 시계열은 수준이 커질수록 변동폭도 함께 커집니다. 이때 로그를 취하면:

$$\log(Y) = \log(T) + \log(S) + \log(I)$$

곱셈이 덧셈으로 바뀔 뿐 아니라, 수학적으로 **로그 함수는 큰 값일수록 압축을 더 세계 하는 성질**이 있습니다(예: $\log(100)=4.6$, $\log(1000)=6.9 \rightarrow 10$ 배 차이가 로그에서는 2.3만큼의 차이로 줄어듦). 그래서 수준이 커서 변동폭도 컸던 구간이 로그를 취하면 상대적으로 눌리면서, **전체 구간에서 변동폭이 비슷해지는 효과**가 생깁니다.

Box-Cox 변환은 로그변환을 일반화한 것으로, 데이터 특성에 맞는 최적의 변환 강도(λ)를 찾아 분산을 더 정교하게 안정화시킵니다.

$$\text{Box-Cox}(Y, \lambda) = (Y^\lambda - 1) / \lambda \quad (\lambda=0 \text{일 때 } \log(Y) \text{와 동일})$$

주의할 점: 로그/Box-Cox 변환은 "분산이 일정해야 한다"는 조건만 고쳐줄 뿐, 평균이 시간에 따라 변하는 추세 문제는 그대로 남아있을 수 있습니다. 그래서 실무에서는 보통 **로그변환 $\rightarrow$ 차분**의 순서로 두 문제를 순차적으로 해결합니다.

3. 추세 제거(Detrending) — 확정적 추세(deterministic trend)를 제거

여기서 중요한 구분이 하나 있습니다. 추세에는 두 종류가 있습니다.

- **확률적 추세(stochastic trend)**: 단위근 때문에 생기는 추세 — 무작위 충격이 누적되며 만들어짐 → **차분으로 제거**
- **확정적 추세(deterministic trend)**: $Y_t = a + b \cdot t + \varepsilon_t$ 처럼 시간(t)에 대한 명확한 함수식으로 표현되는 추세 → **회귀로 추세를 추정해서 빼주는(detrending) 방식으로 제거**

확정적 추세가 있는 경우, 원래 데이터에서 시간(t)을 독립변수로 하는 회귀선(예: $a + b \cdot t$)을 구한 뒤, 원래 값에서 그 회귀선의 예측값을 빼줍니다.

$$\text{잔차(residual)} = Y_t - (\hat{a} + \hat{b} \cdot t)$$

이 잔차에는 "시간이 지날수록 커지는" 성분이 이미 회귀식이 다 설명해서 빠져나갔기 때문에, 남은 부분은 특정 수준(보통 0)을 중심으로 흔들리는 **평균이 일정한 시계열**이 됩니다.

주의할 점: 확정적 추세를 차분으로 제거하려고 하면(과잉차분, over-differencing) 오히려 불필요하게 분산을 키우거나 모형을 왜곡시킬 수 있고, 반대로 확률적 추세(단위근)를 회귀로 제거하려 하면 추세가 완전히 제거되지 않습니다. 그래서 ADF 검정을 할 때 "추세항을 포함한 형태(regression='ct')"로 검정해서, 지금 보이는 추세가 확정적인지 확률적인지 먼저 구분하는 것이 정확한 정상화 방법을 고르는 데 중요합니다.

정리

방법	해결하는 문제	원리
차분	평균이 떠도는 문제 (확률적 추세, 단위근)	누적된 충격에서 새 충격만 남김
로그/Box-Cox	분산이 수준에 비례해 커지는 문제	큰 값일수록 더 세게 압축해 변동폭을 균일화
추세 제거(회귀)	평균이 일정한 함수로 이동하는 문제 (확정적 추세)	추세를 추정해서 원본에서 빼고 잔차만 사용

사용자

아래 내용은 통째로 무슨 말인지 잘 모르겠어. 그림으로 설명해 주면 더 도움이 될지 모르겠지만 이해하기 쉽게 다시 설명해 줘

자기상관 분석

- ACF(자기상관함수, Autocorrelation Function): 시차(lag)에 따른 자기 자신과의 상관관계 — 직접 효과 + 간접 효과 모두 포함
- PACF(편자기상관함수, Partial ACF): 중간 시차의 영향을 통제한 순수한 시차 효과
- 모형 식별에 활용
 - AR(p): PACF 가 p 시차 이후 절단(cut-off), ACF 는 지수적으로 감소
 - MA(q): ACF 가 q 시차 이후 절단, PACF 는 지수적으로 감소
 - ARMA(p,q): 둘 다 점진적으로 감소(tail-off)

Claude

자기상관 분석 부분을 좀 더 쉬운 비유로 먼저 풀어드리고, 그림으로 확인해 보겠습니다.

쉬운 비유로 이해하기

방에서 소리를 지르면 벽에 부딪혀 메아리가 여러 번 반복되다가 잦아드는 상황을 떠올려 보세요.

- **ACF** 는 "지금 소리와 1 초 뒤 메아리가 얼마나 비슷한가, 2 초 뒤는 얼마나 비슷한가..."를 그냥 있는 그대로 켜 것입니다. 문제는 **1 초 뒤 메아리도 사실 2 초 뒤 메아리한테 영향을 주고 있다**는 겁니다. 즉 "오늘↔3 일 뒤"의 관계를 볼 때, 그 안에는 "오늘↔1 일 뒤↔2 일 뒤↔3 일 뒤"로 이어지는 **중간 경로들의 영향**이 다 섞여 들어가 있습니다.

- **PACF** 는 "중간에 낀 애들의 영향은 다 빼고, 딱 이 시차만의 순수한 영향력이 얼마인지"를 잴 것입니다. 즉 "1 일 뒤, 2 일 뒤에 어떤 일이 있었든 무시하고, 오늘이 3 일 뒤에 직접적으로 얼마나 영향을 주는가"만 뽑아낸 것입니다.

모형 식별에 쓰는 이유

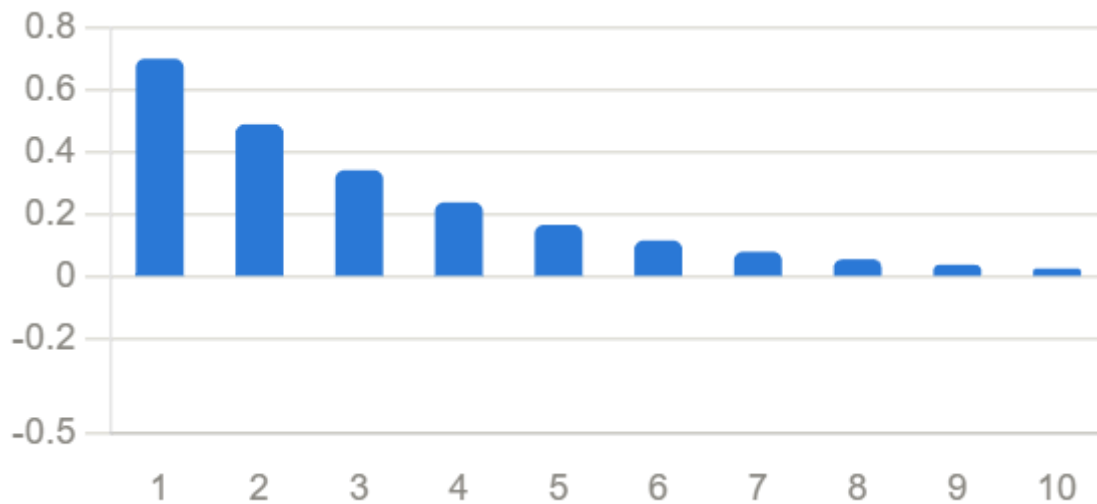
- **AR(p) 모형**: "과거 p 개 시점"이 **직접** 오늘 값을 결정하는 구조입니다. 그래서 PACF 를 보면 딱 p 시차까지만 값이 크고, 그 다음($p+1$ 시차)부터는 **뚝 끊어져(cutoff) 거의 0** 이 됩니다 — 더 이상 직접적인 영향이 없으니까요. 반면 ACF 는 그 영향이 시차를 타고 계속 전파되면서 서서히 잦아드는(tail-off, 지수적 감소) 모습을 보입니다.
- **MA(q) 모형**: 반대로 "최근 q 개의 무작위 충격"만 오늘 값에 섞여 있는 구조입니다. 그래서 ACF 는 q 시차까지만 상관관계가 있고 그 이후는 **뚝 끊어집니다(cutoff)**. PACF 는 오히려 서서히 잦아드는(tail-off) 모습을 보입니다.
- **ARMA(p,q)**: 두 구조가 섞여 있어서 ACF, PACF 둘 다 뚝 끊어지지 않고 **둘 다 서서히 잦아드는** 모습을 보입니다.

즉 요약하면 "어느 쪽이 **뚝 끊기는지**"를 보고 모형을 고르는 것입니다: PACF가 끊기면 AR, ACF가 끊기면 MA, 둘 다 안 끊기면 ARMA.

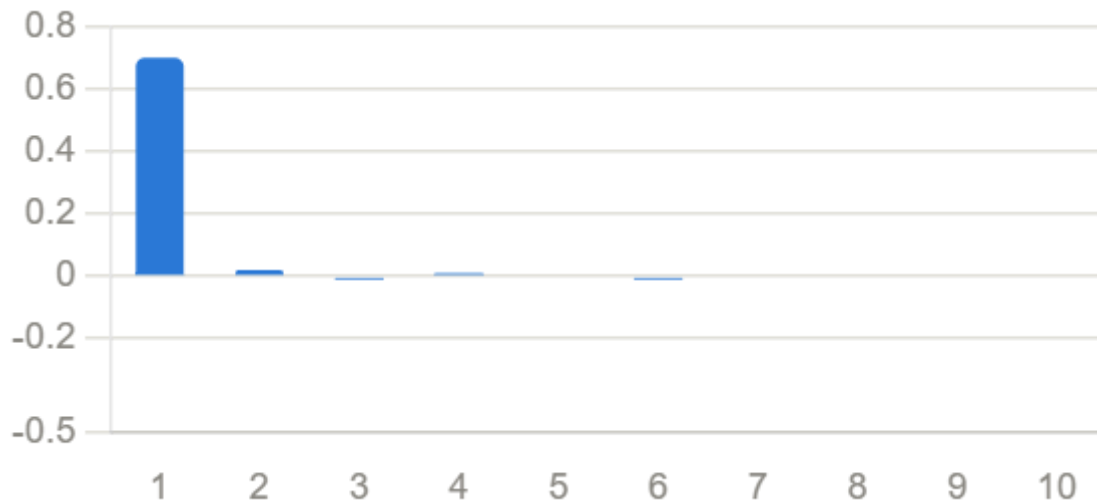
전형적인 패턴을 그림으로 보여드리겠습니다.

[인터랙티브 시각자료 표시됨: "AR(1)과 MA(1) 모형의 전형적인 ACF, PACF 패턴 비교"
— 4개의 막대그래프]

1. AR(1)의 ACF: 서서히 감소 (tail-off)



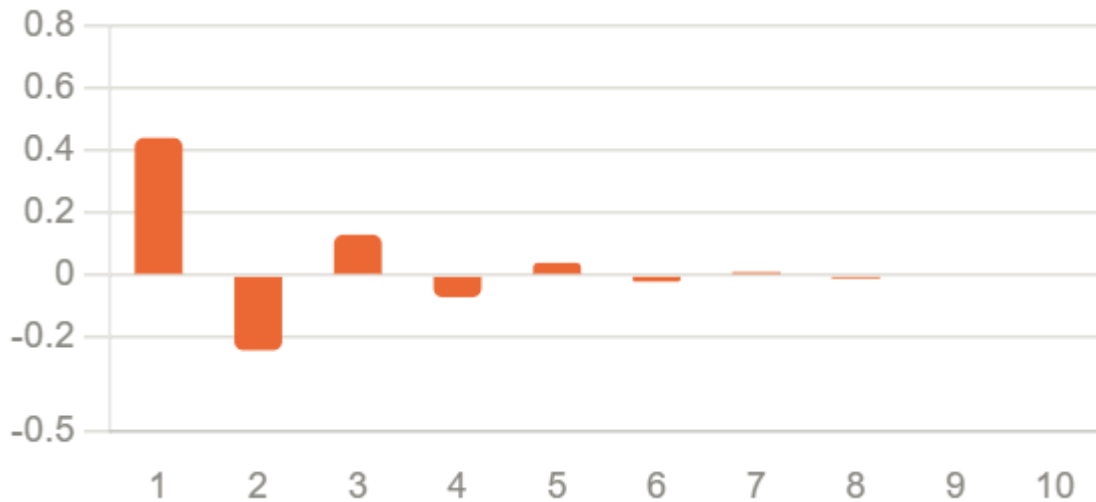
2. AR(1)의 PACF: 1 시차 이후 뚝 끊김 (cutoff)



3. MA(1)의 ACF: 1 시차 이후 뚝 끊김 (cutoff)



4. MA(1)의 PACF: 서서히 감소하며 부호가 번갈아 나타남 (tail-off)



그림에서 보시는 것처럼

- **AR 계열**은 PACF 가 1 시차 이후 거의 0 으로 뚝 떨어지고(파란색 오른쪽), ACF 는 완만하게 계속 줄어드는 모습입니다(파란색 왼쪽)
- **MA 계열**은 정반대로, ACF 가 1 시차 이후 뚝 떨어지고(주황색 왼쪽), PACF 는 부호를 오르내리며 서서히 줄어드는 모습입니다(주황색 오른쪽)

실제 statsmodels의 plot_acf, plot_pacf로 그린 그래프를 보실 때는, 막대 주위에 점선으로 된 신뢰구간(유의수준 경계선)이 표시되는데, **그 점선 안쪽으로 들어온 막대는 "0과**

통계적으로 다르지 않다(=끊긴 것으로 판단)"는 뜻입니다. 그래서 실전에서는 "몇 시차부터 막대가 점선 안으로 쏙 들어가는지"를 보고 p , q 를 결정하시면 됩니다.

사용자

근본적으로 주요 모형이 뭔지 잘 모르겠어. 그림을 통하던가해서 이해하기 쉽게 각각 설명해 줘

모형설명AR(p)자기회귀 — 과거 p 개 관측값의 선형결합으로 현재값 설명MA(q)이동평균 — 과거 q 개 오차항의 선형결합ARMA(p, q)AR + MA 결합, 정상 시계열에 적용

ARIMA(p, d, q)ARMA에 d 차 차분 추가, 비정상 시계열에 적용

SARIMA(p, d, q)(P, D, Q, s)ARIMA + 계절성분(주기 s)SARIMAXSARIMA + 외생변수

(exogenous variable)VAR다변량 시계열, 여러 변수가 서로 영향을 주고받을 때지수평활

(Exponential Smoothing)단순지수평활(수준만) → Holt(추세 추가) → Holt-Winters(계절성까지 추가)

Claude

각 모형이 왜 필요한지, 어떤 상황에 쓰는지를 먼저 쉬운 예시로 설명해 드리고, 모형들이 서로 어떻게 쌓여서 발전하는지 구조를 그림으로 보여드리겠습니다.

하나씩 쉬운 예시로 이해하기

AR(p) — 자기회귀: "오늘 기온은 어제, 그제 기온과 비슷한 경향이 있다"처럼, **과거 값 자체**가 오늘 값에 영향을 주는 구조입니다. 예: "어제 매출이 높았으면 오늘도 높을 가능성이 크다(관성/모멘텀)"

MA(q) — 이동평균: 과거 "값"이 아니라 과거 "예상치 못한 충격(오차)"이 남아서 영향을 주는 구조입니다. 예: "지난주에 갑자기 터진 뉴스(예측 못 한 사건)의 여파가 이번 주까지 조금 남아있다"

ARMA(p, q): 위 두 가지 원인(관성 + 충격의 여운)이 동시에 작용하는 경우. 단, 이 모형은 **정상 시계열에만** 쓸 수 있습니다(추세가 없어야 함).

ARIMA(p, d, q): 실제 데이터는 추세가 있어 비정상인 경우가 많으므로, " d 번 차분해서 정상으로 만든 뒤 ARMA를 적용"하는 실전용 모형입니다. 예: 꾸준히 성장하는 회사의

매출 예측

SARIMA(p,d,q)(P,D,Q,s): ARIMA에 "계절성(예: 매년 12월에 반복되는 패턴)"까지 추가로 반영. 예: 에어컨 판매량(매년 여름 급증)

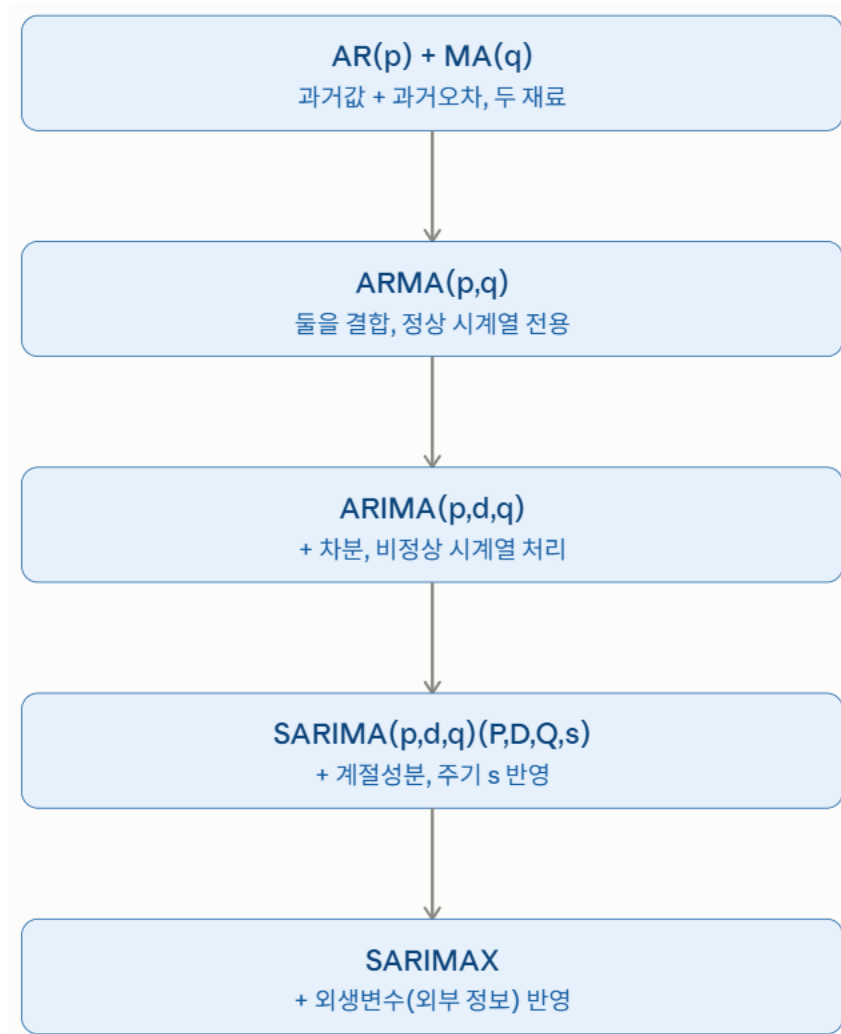
SARIMAX: SARIMA에 "외부 변수(외생변수)"까지 반영. 예: 아이스크림 판매량을 예측할 때 계절성뿐 아니라 "그날의 실제 기온"이라는 외부 정보도 같이 사용

VAR: 지금까지는 변수 하나만 봤다면, VAR는 **여러 변수가 서로 영향을 주고받는** 경우입니다. 예: "금리가 오르면 환율에 영향을 주고, 환율은 다시 수출에 영향을 주고, 수출은 다시 금리 정책에 영향을 준다"처럼 여러 지표를 동시에 모델링

지수평활(Exponential Smoothing): ARIMA와는 다른 접근으로, "최근 데이터에 더 큰 가중치를 주는 가중평균"으로 예측합니다. 단순(수준만) → Holt(추세도 반영) → Holt-Winters(계절성까지 반영) 순으로 점점 정교해집니다.

이 모형들이 서로 어떻게 확장되어 가는지 구조를 그림으로 보여드리겠습니다.

[인터랙티브 시각자료 표시됨: "ARIMA 계열 모형의 발전 구조" — 세로 방향 5단계 다이어그램]



보시는 것처럼 매 단계마다 "이전 모형이 못 다루던 문제 하나씩"을 해결하며 확장됩니다: ARMA는 정상 시계열만 다루니 → 차분을 더해 ARIMA로 비정상까지, 계절 패턴이 있으니 → SARIMA로 계절성까지, 외부 정보(기온, 프로모션 여부 등)도 반영하고 싶으니 → SARIMAX로 발전하는 흐름입니다.

지수평활법도 같은 방식으로 발전합니다.

[인터랙티브 시각자료 표시됨: "지수평활법의 발전 구조" — 세로 방향 3단계 다이어그램]



참고: VAR 는 별개의 확장 방향

위 두 흐름은 모두 "변수 하나"를 다루는 모형들의 발전 과정입니다. **VAR**는 이 흐름과는 성격이 다른데, "변수가 여러 개이고 서로 영향을 주고받는" 경우에 쓰는 모형입니다. 굳이 비유하면 위 흐름이 "한 사람의 습관을 점점 더 정교하게 예측하는 방법"이라면, VAR는 "여러 사람이 서로 영향을 주고받는 관계망 전체를 동시에 예측하는 방법"에 가깝습니다.

실전에서 어떤 모형을 고르는지 요약하면

- 데이터가 하나의 변수 + 추세 없음 → **ARMA**
- 하나의 변수 + 추세 있음(비정상) → **ARIMA**
- 하나의 변수 + 추세 + 계절성 → **SARIMA**
- 위에 더해 외부 정보(날씨, 프로모션 등)까지 활용 가능 → **SARIMAX**
- 여러 변수가 서로 영향을 주고받음 → **VAR**
- 복잡한 통계적 가정 없이 최근 데이터 위주로 빠르게 예측하고 싶음 → **지수평활 계열**

사용자

아래 모형 선택 기준에서 AIC, BIC가 뭔지 자세히 설명해 주고 왜 낮을 수록 좋으며 실제 여러 (p,d,q) 조합 중 AIC/BIC 최소인 모형 선택 (Grid search)하는 예를 보여 줘

모형 선택 기준

- AIC(Akaike Information Criterion), BIC(Bayesian Information Criterion):
낮을수록 좋음, BIC 가 더 강한 복잡도 페널티
- 여러 (p,d,q) 조합 중 AIC/BIC 최소인 모형 선택 (Grid search)

Claude

AIC와 BIC는 둘 다 "**모형이 데이터를 얼마나 잘 설명하는가**"와 "**모형이 얼마나 복잡한가(파라미터가 많은가)**"를 동시에 고려해서, 이 둘의 균형점을 찾기 위한 지표입니다.

왜 이런 지표가 필요한가

모형에 파라미터(p, q 등)를 무작정 늘리면 데이터에 대한 적합도(likelihood, 우도)는 항상 좋아집니다 — 파라미터가 많을수록 학습 데이터의 세세한 잡음까지 다 따라갈 수 있으니까요. 하지만 이건 **과적합(overfitting)**으로 이어져서, 정작 미래 예측에는 오히려 성능이 나빠집니다. 그래서 "적합도가 좋아지는 만큼, 복잡해진 대가로 페널티도 부과" 하는 지표가 필요한데, 그게 AIC와 BIC입니다.

수식과 각 항의 의미

$$AIC = -2 \cdot \ln(L) + 2k$$

$$BIC = -2 \cdot \ln(L) + k \cdot \ln(n)$$

- **L**: 우도(likelihood) — 이 모형이 실제 관측된 데이터를 얼마나 잘 설명하는지에 대한 확률적 점수. L 이 클수록(잘 맞을수록) $-2 \cdot \ln(L)$ 은 **작아짐**
- **k**: 모형의 파라미터 개수 (ARIMA(p,d,q)라면 대략 p+q 개 수준)
- **n**: 관측치(데이터) 개수

각 수식은 두 부분의 합입니다.

- **$-2 \cdot \ln(L)$ (적합도 항)**: 모형이 데이터를 잘 설명할수록 작아짐 — "얼마나 못 맞췄는가"에 대한 벌점

- $2k$ 또는 $k \cdot \ln(n)$ (복잡도 페널티 항): 파라미터가 많아질수록 커짐 — "얼마나 복잡한가"에 대한 벌점

왜 낮을수록 좋은가

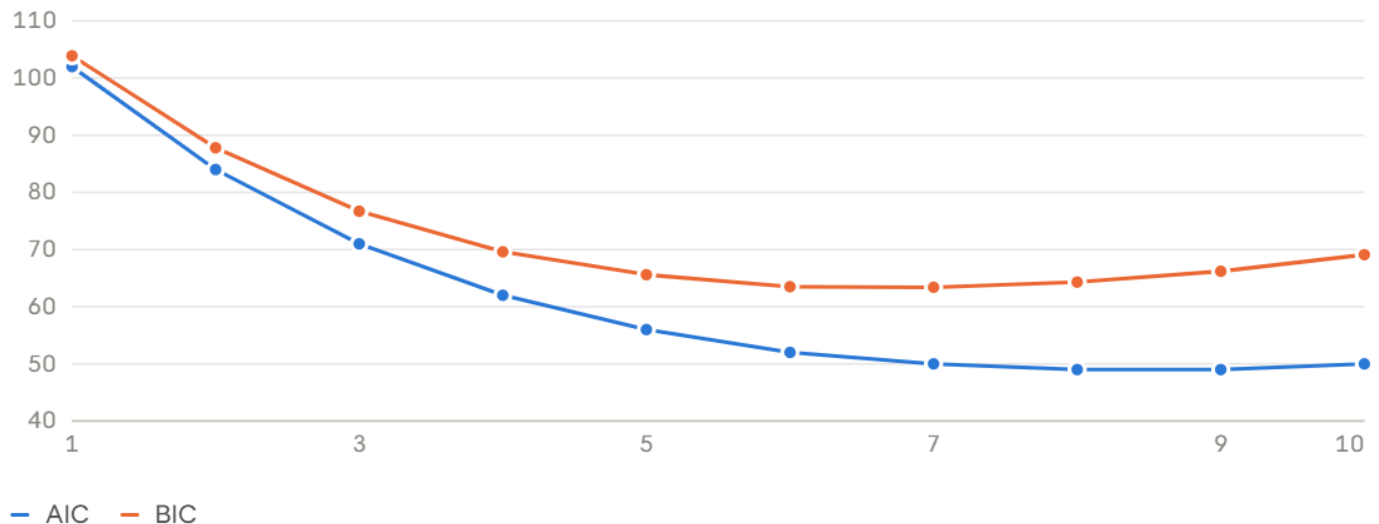
두 항 모두 "나쁜 것"에 대한 벌점이기 때문입니다. 적합도가 나쁘면 첫째 항이 커지고, 모형이 지나치게 복잡하면 둘째 항이 커집니다. 그래서 **AIC/BIC가 낮다는 건 "적당히 잘 맞으면서도 지나치게 복잡하지 않은, 효율적인 모형"**이라는 뜻입니다. 파라미터를 늘려서 적합도가 좋아져도, 그만큼 페널티 항이 늘어나기 때문에 무작정 파라미터를 늘리는 게 항상 유리하지는 않게 되는 구조입니다.

파라미터 개수(k)가 늘어남에 따라 적합도 항은 계속 줄지만 페널티 항은 계속 늘어나서, 둘을 더한 AIC/BIC는 보통 U자 형태를 그리며 어느 지점에서 최소값을 가집니다.

[차트 표시됨: "파라미터 개수에 따른 AIC / BIC 변화 (예시)" — 라인 차트, x축: 파라미터 개수(k) 1~10, y축: AIC/BIC 값]

파라미터 개수에 따른 AIC / BIC 변화 (예시)

파라미터 개수(k)별 AIC / BIC 값



그래프를 보시면 두 지표 모두 파라미터가 늘어날수록 처음엔 줄어들다가(적합도 개선 효과가 큼), 어느 지점을 지나면 다시 늘어나는(복잡도 페널티가 더 커짐) U자 형태입니다. 그리고 **BIC의 최소점이 AIC보다 더 왼쪽(적은 파라미터 쪽)에서 나타나는** 걸 보실

수 있는데, 이게 "BIC가 더 강한 복잡도 페널티를 가진다"는 말의 실제 의미입니다.

AIC 와 BIC 의 페널티 차이

- AIC 의 페널티: $2k$ — 파라미터 하나 늘 때마다 고정적으로 2 씩 증가
- BIC 의 페널티: $k \cdot \ln(n)$ — 관측치 개수 n 이 많아질수록 페널티가 더 세짐

데이터가 웬만큼 크면($n \geq 8$ 정도만 되어도 $\ln(n) > 2$) BIC의 페널티가 AIC보다 항상 더 강합니다. 그래서 **BIC는 AIC보다 더 단순한(파라미터 적은) 모델을 선호**하는 경향이 있습니다. 실무에서는 "예측 성능 위주면 AIC, 모형의 단순성·해석 가능성도 중요하면 BIC"를 참고 지표로 같이 보는 경우가 많습니다.

Grid Search 실전 코드 예시

여러 (p,d,q) 조합을 모두 시도해보고 AIC/BIC가 가장 낮은 조합을 자동으로 찾는 코드입니다.

```
import itertools
import pandas as pd
from statsmodels.tsa.arima.model import ARIMA
import warnings
warnings.filterwarnings('ignore')
# 탐색할 파라미터 범위 설정
p_range = range(0, 4) # 0, 1, 2, 3
d_range = range(0, 2) # 0, 1
q_range = range(0, 4) # 0, 1, 2, 3

results = []

for p, d, q in itertools.product(p_range, d_range, q_range):
    try:
```

```

model = ARIMA(df['value'], order=(p, d, q))
fitted = model.fit()
results.append({
    'order': (p, d, q),
    'AIC': fitted.aic,
    'BIC': fitted.bic
})
except Exception as e:
    # 수렴 실패 등으로 적합이 안 되는 조합은 건너뛴다
    continue

results_df = pd.DataFrame(results).sort_values('AIC').reset_index(drop=True)
print(results_df.head(10))

# AIC 기준 최적 모형
best_aic = results_df.loc[results_df['AIC'].idxmin()]
print(f"AIC 기준 최적 모형: order={best_aic['order']}, AIC={best_aic['AIC']:.2f}")

# BIC 기준 최적 모형
best_bic = results_df.loc[results_df['BIC'].idxmin()]
print(f"BIC 기준 최적 모형: order={best_bic['order']}, BIC={best_bic['BIC']:.2f}")

```

출력 예시

	order	AIC	BIC
0	(2, 1, 2)	842.31	858.47
1	(1, 1, 2)	843.05	856.18
2	(2, 1, 1)	844.12	857.25
3	(3, 1, 2)	844.88	864.09
...			

AIC 기준 최적 모형: order=(2, 1, 2), AIC=842.31

BIC 기준 최적 모형: order=(1, 1, 2), BIC=856.18

위 결과처럼 실제로 **AIC와 BIC가 서로 다른 조합을 최적으로 뽑는 경우가** 흔합니다. AIC는 적합도를 좀 더 중시해서 (2,1,2)처럼 파라미터가 조금 더 많은 모형을 고르고, BIC는 페널티가 강해서 (1,1,2)처럼 더 단순한 모형을 고르는 식입니다.

자동화 도구: auto_arima 사용

앞서 알려드린 pmdarima의 auto_arima는 이 grid search 과정을 내부적으로 더 효율적으로(모든 조합을 다 시도하지 않고 단계적으로 탐색) 수행해 줍니다.

```
from pmdarima import auto_arima

auto_model = auto_arima(
    df['value'],
    start_p=0, max_p=3,
    d=None,          # 자동으로 차분 차수 결정 (ADF 검정 기반)
    start_q=0, max_q=3,
    information_criterion='aic', # 'bic'로 바꾸면 BIC 기준으로 탐색
    stepwise=True,
    trace=True       # 탐색 과정을 로그로 출력
)

print(auto_model.summary())
print(f"최적 order: {auto_model.order}")
```

실기 시험 팁: 시험 문제에서 "최적 모형을 선택하고 그 근거를 설명하라"는 문항이 나오면, 단순히 auto_arima 결과만 적지 말고, "AIC/BIC를 비교했을 때 어떤 값이 더 낮아서 이 조합을 선택했다"는 **비교 근거를 표(위 `results\_df`)와 함께 서술**하시면 채점에 유리합니다.